

Rapport ODAT

Oleg PAILLOT, Elliot BURNS

21 Janvier 2025

Table des matières

1	SDL	3
1.1	Introduction	3
1.2	Environnement	3
1.2.1	Process de base	5
1.2.2	Types	5
1.2.3	Signaux	6
1.2.4	Canaux	7
1.3	Process	7
1.3.1	pSupervisor	7
1.3.2	pPickupRobot	14
1.3.3	pDropoffRobot	15
1.3.4	pCentral	16
1.3.5	pTable	17
1.4	Simulation	18
1.4.1	Cas normal	18
1.4.2	Erreur d'alimentation	25
1.4.3	Erreur d'usinage	27
1.4.4	Machine en panne	29
1.4.5	Erreur de retrait	31
1.5	Conclusion	33
2	Ordonnancement Temps Réel	33
2.1	Introduction	33
2.2	Mode 1 : Tâches Périodiques Indépendantes	34
2.2.1	Présentation	34
2.2.2	Ensembles de Tâches	34
2.2.3	Ordonnancement avec RM	34
2.2.4	Calcul de la Charge Cumulative (CS) pour Exemple 2	36
2.2.5	Calculs CNS pour l'Ordonnancabilité RM	37
2.2.6	Conclusion	38
2.3	Ordonnancement avec EDF	38
2.3.1	Présentation	38

2.3.2	Calcul pour l'Exemple 1	38
2.3.3	Calcul pour l'Exemple 2	39
2.4	Mode 2 : Tâches Périodiques et Apériodiques	39
2.4.1	Présentation	39
2.4.2	Serveur à Scrutation	40
2.5	Mode 3 : Contraintes de Précédence	41
2.5.1	Présentation	41
2.5.2	Ordonnancement avec RM	42
2.5.3	Ordonnancement avec EDF	42
2.6	Mode 4 : Partage de Ressources	43
2.6.1	Présentation	43
2.7	Analyse Assistée	44
2.7.1	Présentation de l'Outil	44
2.7.2	Résultats de l'Analyse	44
2.8	Conclusion	45
Table des figures		46

1 SDL

1.1 Introduction

Ce rapport traite de la conception et de la spécification d'un système de fabrication automatisé comprenant un convoyeur, des robots, et des machines d'usinage, pilotés par un superviseur central. Le système est modélisé en SDL pour capturer son fonctionnement parallèle et synchronisé, avec prise en compte des contraintes temps réel pour gérer anomalies et pannes.

1.2 Environnement

Afin de mettre en place le système, il faut définir son environnement, c'est-à-dire les process, les signaux échangés entre eux et les différents canaux par lesquels ces derniers communiqueront. Par ailleurs, différents types ont été déclarés afin d'aider au développement des process. Nous détaillerons l'utilité de chacun de ces types mais aussi des signaux, toutefois en commençant d'abord par une explication des process.

```

/* Max number of tables */
synonym NUM_TABLE Integer = 5;

/* Type for table number (useful for limiting it) */
syntype TableNumberType = Integer
  default 1;
  constants 1..NUM_TABLE
endsyntype;

newtype Specialization
  literals Fraisage, Percage, Tournage;
endnewtype;

newtype SpeArray
  Array(TableNumberType, Specialization)
endnewtype;

/* Struct for a table entry in the array for the supervisor */
newtype TableEntry struct
  tableID Pid;
  specialization Specialization;
endnewtype;

newtype pTableArray
  Array(TableNumberType, TableEntry)
endnewtype;

newtype OccupiedArray
  Array(Pid, Boolean)
endnewtype;

newtype Item struct
  ID Integer;
  specialization Specialization;
endnewtype;

newtype waitingItemBag
  Bag(Item)
endnewtype;

newtype workingItemBag
  Bag(Item)
endnewtype;

```

```

signal sNewGood(Item);
signal sDropoffRequest(Item), sDropoffFinished(Item);
signal sTableRequest(Pid, Item), sTableFinished(Pid, Item), sTableWorking(Pid), sTableDrop(Pid);
signal sPickupRequest(Item), sPickupFinished(Item);
signal sSensor(Item), sConveyorClear;
signal sError, sDone, sOutOfOrder(Pid);

```

FIGURE 1 – Déclarations

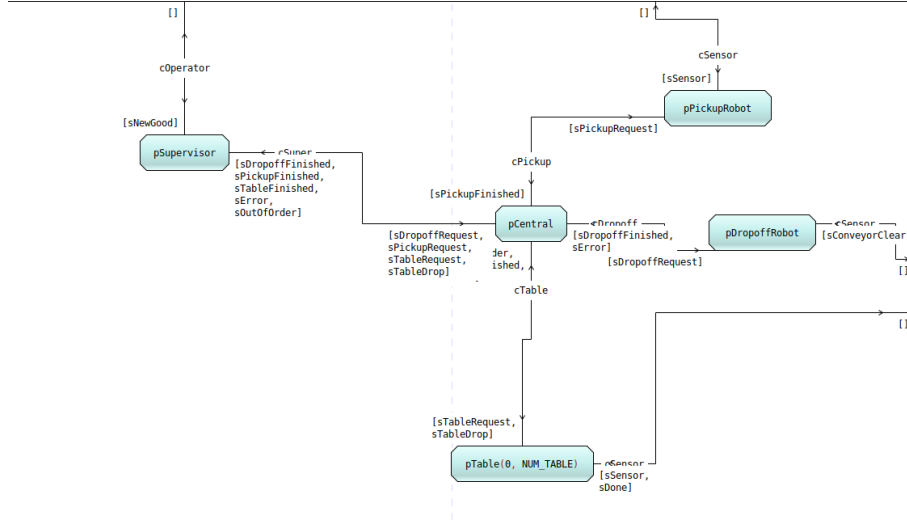


FIGURE 2 – Process et canaux

1.2.1 Process de base

En suivant l'énoncé, nous pouvons retrouver 5 process correspondant aux différents éléments du système, tous visibles dans la figure 2 :

1. **pSupervisor** : le superviseur du système, contrôlant la chronologie des événements
2. **pCentral** : le bloc de communication central, ne servant qu'à relayer les signaux entre les différents acteurs
3. **pDropoffRobot** : le robot d'alimentation, déposant les pièces demandées par le superviseur et indirectement l'opérateur sur le tapis roulant
4. **pPickupRobot** : le robot de retrait, récupérant les pièces usinées par les machines déposées sur le tapis roulant
5. **pTable** : une table d'usinage d'une certaine spécialisation, pouvant ainsi traiter les différentes pièces qui lui sont destinées

1.2.2 Types

Certains types ne sont que des composants d'autres types, ne servant qu'à les mettre en place afin de faire fonctionner le système. Nous aborderons ceux qui sont réellement utiles à notre implémentation dans cette section, mais ils sont bien sûr tous visibles et disponibles dans la figure 1.

Le premier type important est **Specialization**, qui permet définir la spécialisation de chaque table. Les trois spécialisations possibles sont : Fraisage, Perçage et Tournage. Grâce à ceci, une table est donc définie par son PID et sa spécialisation, regroupés dans le type **TableEntry** afin d'être mis dans des arrays ou encore des bags.

Par ailleurs, le type `Item`, qui représente les différents objets devant être usinés, utilise aussi `Specialization` pour que les capteurs des robots puissent reconnaître si un item leur est attribué. En plus de cela, un simple entier est utilisé comme ID pour aider à l'identification des objets.

Enfin `OccupiedArray` est un simple array associatif entre tables et booléens afin de garder en mémoire si une table d'usinage est actuellement occupée ou non.

1.2.3 Signaux

Globalement, les signaux sont répartis entre les différents process et ont des noms permettant de facilement reconnaître leur utilité. Les quelques exceptions à cette règle sont souvent des signaux spécifiques ayant pour origine l'utilisateur `RTDS_Env` ou des signaux d'erreur.

Pour introduire un objet dans le système, le signal `sNewGood(Item)` est envoyé au superviseur par `RTDS_Env` avec un `Item` customisé selon les souhaits de l'opérateur.

Concernant le robot d'alimentation, deux signaux sont utilisés : `sDropoffRequest(Item)` et `sDropoffFinished(Item)`. Le premier arrive en entrée du process depuis le superviseur afin que le robot se prépare à déposer la pièce. Le second est en sortie du process et annonce au superviseur qu'il a fini sa tâche.

Les différentes tables d'usinage disposent du plus grand nombre de signaux, décomposant les tâches effectuées par celles-ci. Le premier signal, `sTableRequest(Pid,Item)` provient du superviseur et permet de préparer la table à recevoir un objet. Une fois qu'elle a fini son usinage, elle attend `sTableDrop(Pid)` afin de déposer l'objet sur le tapis roulant tout en prévenant le superviseur avec `sTableFinished(Pid, Item)`. Le signal `sTableWorking(Pid)` est un ancien signal déprécié.

Le robot de retrait est un miroir du robot d'alimentation, avec un signal `sPickupRequest(Item)` permettant de préparer le robot à la récupération d'un objet du tapis roulant et le signal `sPickupFinished(Item)` signalant la fin de son opération.

A partir de maintenant, les signaux dont nous allons parler proviennent exclusivement de `RTDS_Env` et permettent de faire avancer le système dans son fonctionnement. Ils permettent d'émuler, par exemple, les capteurs des tables et celui du robot de retrait avec `sSensor(Item)` ou encore avoir le retour du tapis roulant afin de savoir s'il est capable d'accueillir une autre pièce avec `sConveyorClear`. Par ailleurs, un signal spécial `sDone` est utilisé pour simuler la fin du travail d'une table. Ces deux derniers signaux ont été rajoutés pour simuler les cas d'erreurs demandés dans la partie 2 de l'énoncé.

Pour conclure sur les signaux, `sError` est un simple signal d'erreur permettant de mettre en défaillance le superviseur, et donc le système, lors des cas vus dans la partie 2 de l'énoncé. `sOutOfOrder(Pid)` est exclusivement utilisé par les tables pour signaler au superviseur qu'elle est défaillante, sans pour autant mettre en déroute le système.

1.2.4 Canaux

Afin de faire véhiculer ces différents signaux, plusieurs canaux ont été créés. Etant donné que tout est regroupé sous le **pCentral**, il est le process avec le plus de signaux et d'entrées-sorties du système. Il est difficile de voir correctement les signaux sur la figure 2, la sous-sous-section 1.2.3 est de ce fait pratique pour connaître le sens des signaux.

Voici tout de même la liste des différents canaux :

1. **cOperator** : canal pour la communication entre l'opérateur et le **pSupervisor**
2. **cSuper** : canal pour la communication entre **pCentral** et **pSupervisor**, regroupe énormément de signaux
3. **cTable** : canal entre les différentes tables et **pCentral**
4. **cDropoff** : canal entre le robot d'alimentation et **pCentral**
5. **cPickup** : canal entre le robot de retrait et **pCentral**
6. **cSensor** : les différents canaux entre les robots disposant de capteurs ou nécessitant des informations non accessibles dans le système actuel et **RTDS_Env**

1.3 Process

Nous explorerons dans cette section les différents process et leurs transitions d'états nécessaires au bon fonctionnement du système.

1.3.1 pSupervisor

pSupervisor est probablement le process le plus compliqué et poussé de notre système. Il se sépare en plusieurs parties, que ce soit pour la gestion des objets, des erreurs ou des différentes tables disponibles.

```
dcl
waiting waitingItemBag,
working workingItemBag,
usedTable Pid,
i Integer,
readyDrop Boolean := TRUE,
readyPickup Boolean := TRUE,
tableFound Boolean := FALSE,
newItem, droppedItem, pickedItem, finishedItem, randomItem Item,
tablePID Pid;
```

FIGURE 3 – Déclarations du superviseur

Dans la figure 3 nous pouvons voir les différentes déclarations essentielles aux futures transitions, nous retrouvons dedans les différents types que nous avons créés auparavant.

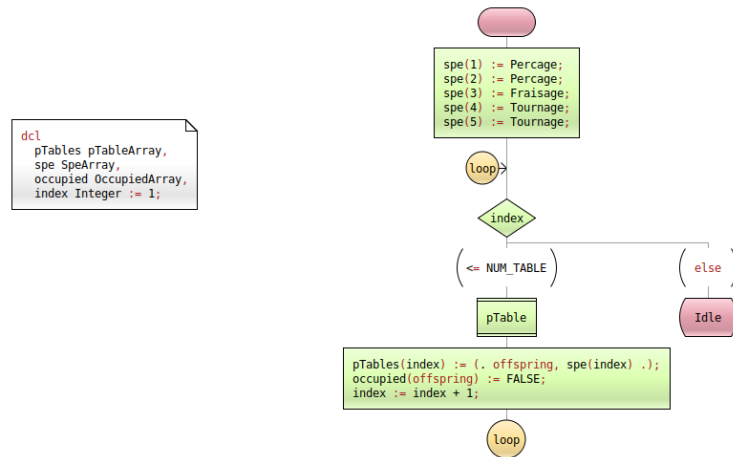


FIGURE 4 – Création des tables

Avant de faire quoi que ce soit avec les tables, il faut les créer dynamiquement et enregistrer leur PID et spécialisation, ceci est réalisé par la boucle visible dans la figure ci-dessus.

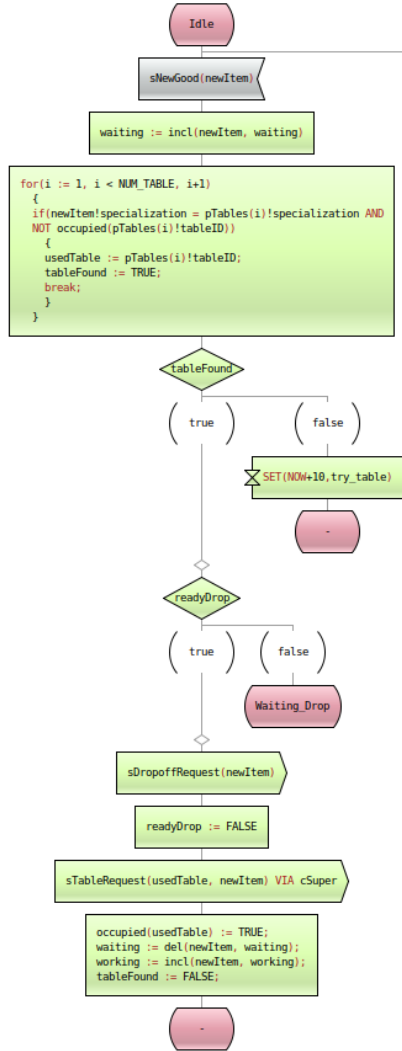


FIGURE 5 – Gestion des nouveaux objets

Lors de la réception d'une demande d'un nouvel objet de la part de l'opérateur, le superviseur commence par rajouter cet objet dans **waiting**, un **Bag** regroupant les objets pas encore traités par une table. Ensuite, une boucle **for** est utilisée pour vérifier la possibilité d'attribuer cet objet à la table ayant la spécialisation correspondante et qui est bien sûr libre. Nous pouvons vérifier cela en passant par l'array associatif **occupied**, contenant les PID des tables et un booléen.

S'ensuit un simple **if** permettant de savoir si la boucle **for** précédente a pu trouver une table, si oui nous continuons normalement le protocole, sinon un

timer est lancé afin de refaire une vérification plus tard. Un autre if est présent pour savoir si le robot d'alimentation est disponible, sans quoi nous passons dans un état d'attente qui sera détaillé plus tard.

Dans le meilleur des cas, une table est trouvée et le robot est libre, nous envoyons donc un signal de "request" à la table et au robot d'alimentation et nous mettons à jour les différents arrays et bags contenant les objets et les tables.

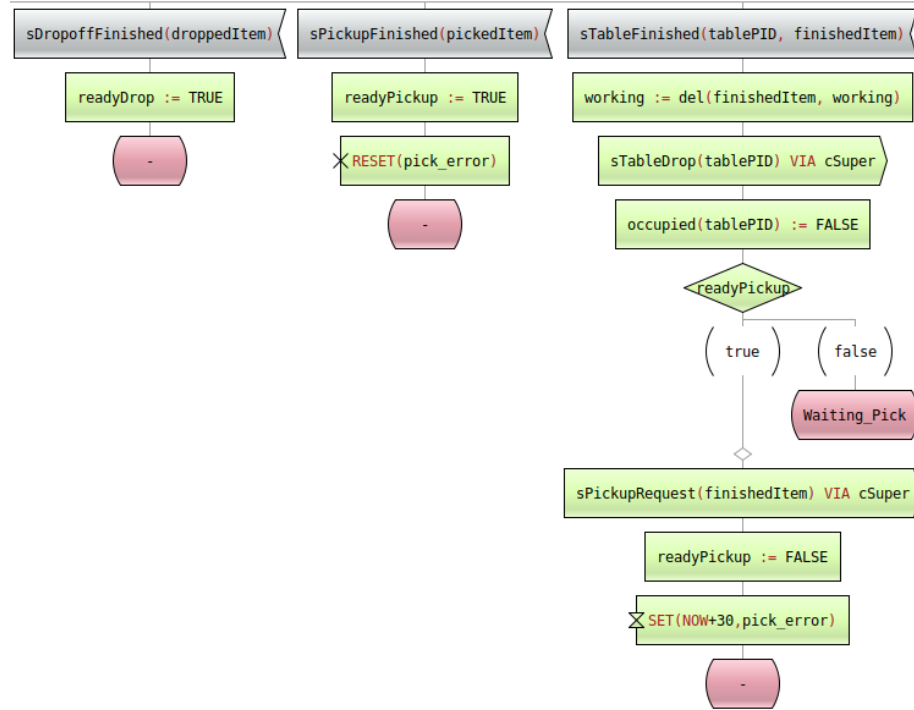


FIGURE 6 – Gestion des signaux provenant des robots

Le superviseur gère bien sûr les signaux de fin de cycle des différents robots et réagit de façon appropriée. Que ce soit pour le robot d'alimentation ou de retrait, nous mettons à jour le booléen correspondant pour les prochains checks, celui de retrait reset aussi un timer qui est set justement dans le dernier signal visible dans la figure 6, **pick_error**. Lorsqu'une table termine son usinage, nous mettons à jour les arrays et bags puis nous lui envoyons un signal pour qu'elle pose la pièce sur le tapis roulant.

Si le robot de retrait est occupé, nous allons dans un état d'attente. Sinon, nous pouvons directement lui envoyer une request et lancer le timer **pick_error**. Ce dernier est une demande de la partie 2, où si jamais le robot de retrait n'arrive

pas à prendre un objet du tapis roulant pour une quelconque raison dans un espace de 30 secondes, le système doit être mis en défaillance.

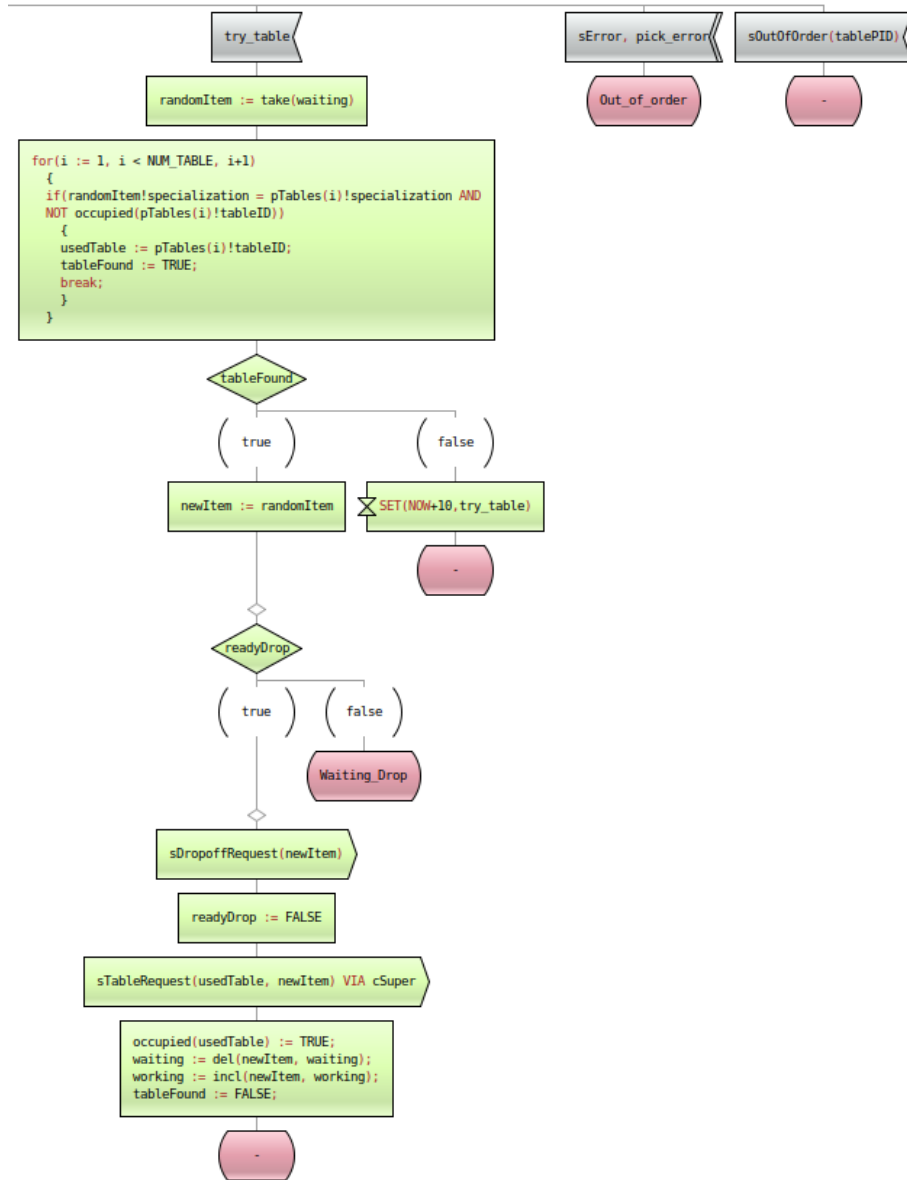


FIGURE 7 – Recheck des tables

Comme dit précédemment, un nouveau check est réalisé toutes les 10 secondes dans le cas où un nouvel objet ne trouve pas sa table correspondante.

La structure est globalement la même que le check initial après le signal de l'opérateur, la différence principale étant que nous prenons un objet au hasard du **Bag** contenant les différents objets en attente de trouver leur table.

Par ailleurs, nous pouvons voir l'état de défaillance qui est activé en fonction de **sError** et **pick_error**. Une fois dans cet état, il n'y a aucun moyen d'en sortir, imitant ainsi un blocage nécessitant l'intervention d'un technicien ou de l'opérateur. Le signal **sOutOfOrder(tablePID)** qui permet de mettre une table en panne, est en réalité suivi d'aucun traitement. En effet, une table en panne n'enverra plus aucun signal et ne se met en panne qu'avant le signal **sTableFinished**, il nous suffit donc de la laisser dans l'array **occupied** afin qu'elle ne soit plus utilisée.

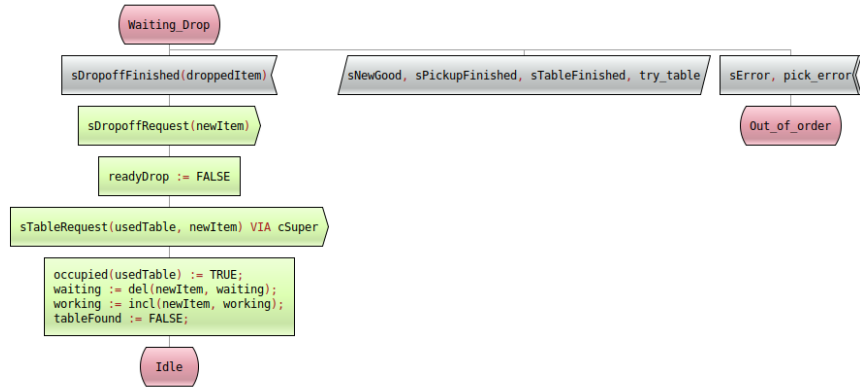


FIGURE 8 – Attente du robot d'alimentation

Dans cet état d'attente, nous copions globalement les dernières transitions utilisées dans le cas d'un nouvel objet, nous attendons juste que le robot d'alimentation finisse son travail. Les autres signaux sont sauvegardés et seront traités en priorité lors du retour à l'état **Idle**, notez cependant qu'un signal d'erreur sera traité immédiatement.

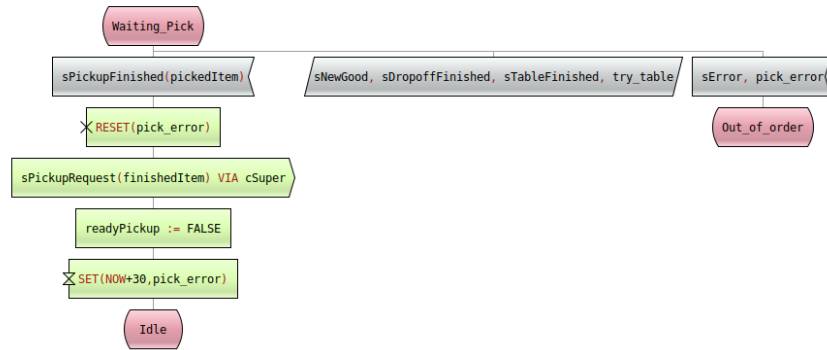


FIGURE 9 – Attente du robot de retrait

Idem dans le cas de l'attente du robot de retrait, là aussi nous copions globalement la logique d'auparavant et nous sauvegardons les signaux.

1.3.2 pPickupRobot

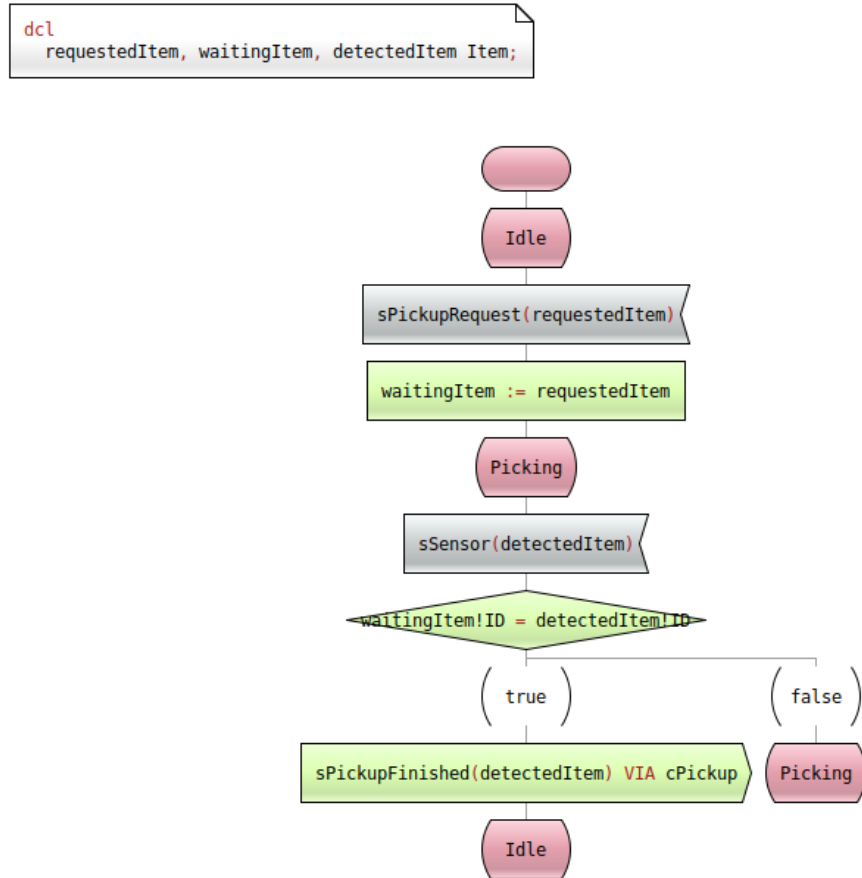


FIGURE 10 – Robot de retrait

Le robot de retrait est relativement simple à comprendre, si un objet est demandé à être retiré, il l'enregistre et attend que son capteur le détecte. Tout autre objet que celui demandé sera ignoré, jusqu'à qu'éventuellement l'objet attendu arrive et soit retiré. Un signal annonçant la fin de son travail sera envoyé au superviseur.

1.3.3 pDropoffRobot

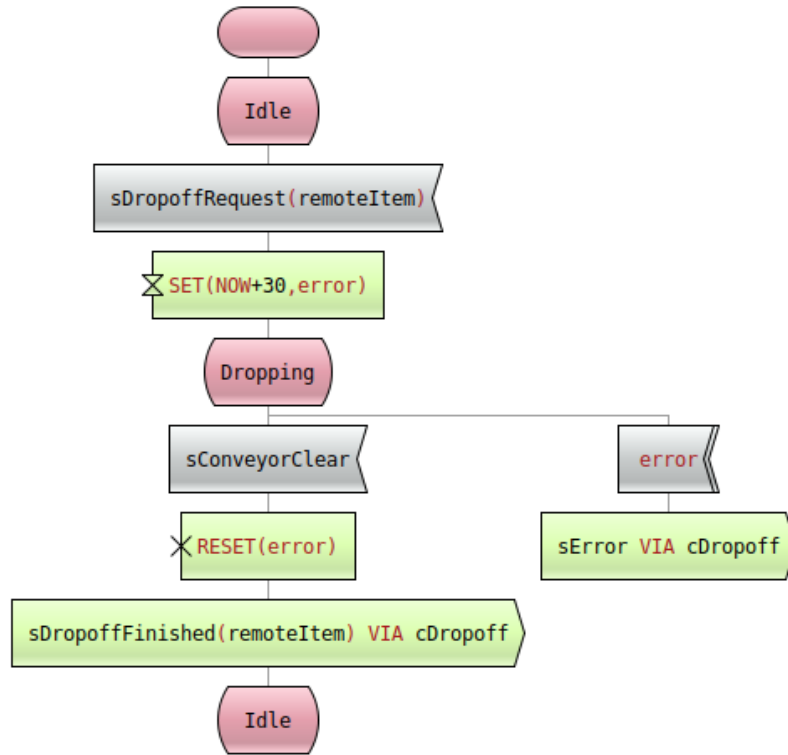


FIGURE 11 – Robot d'alimentation

Le robot d'alimentation diffère légèrement du robot de retrait. En premier lieu, le timer pour gérer les 30 secondes demandé dans la partie 2 de l'énoncé est inclus directement dans le process. Dès lors qu'une request est reçue, le timer est lancé et le process enverra éventuellement un signal d'erreur, seulement si le tapis roulant n'est pas libre pour pouvoir poser l'objet demandé par le superviseur. Autrement, l'objet est tout simplement déposé puis un signal est envoyé au superviseur.

1.3.4 pCentral

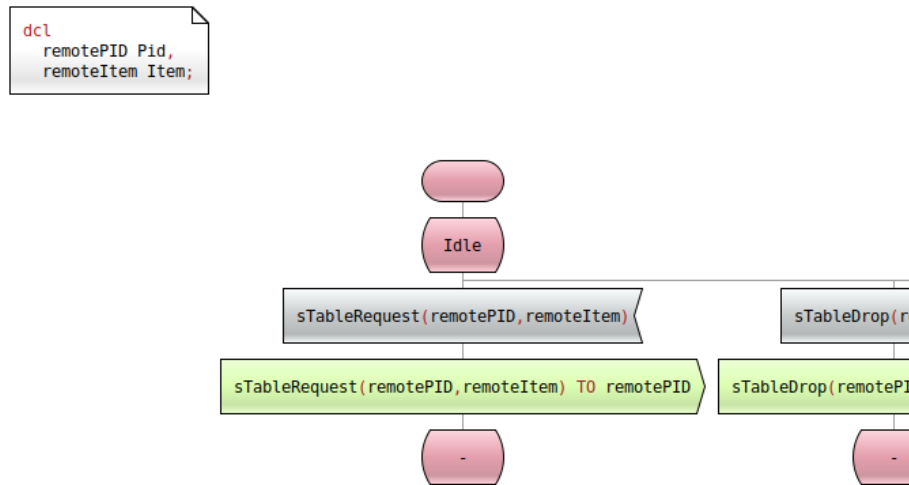


FIGURE 12 – Bloc de communication central

Le bloc de communication central est simple dans son fonctionnement, lors de la réception d'un signal, il redirige tout simplement le signal au destinataire. Il n'y a pas vraiment grand chose à dire quant à ce process en l'état, mais il peut se révéler potentiellement pratique le moment où de nouveaux process avec certaines capacités devront être rajoutés. On découple ainsi la communication du superviseur.

1.3.5 pTable

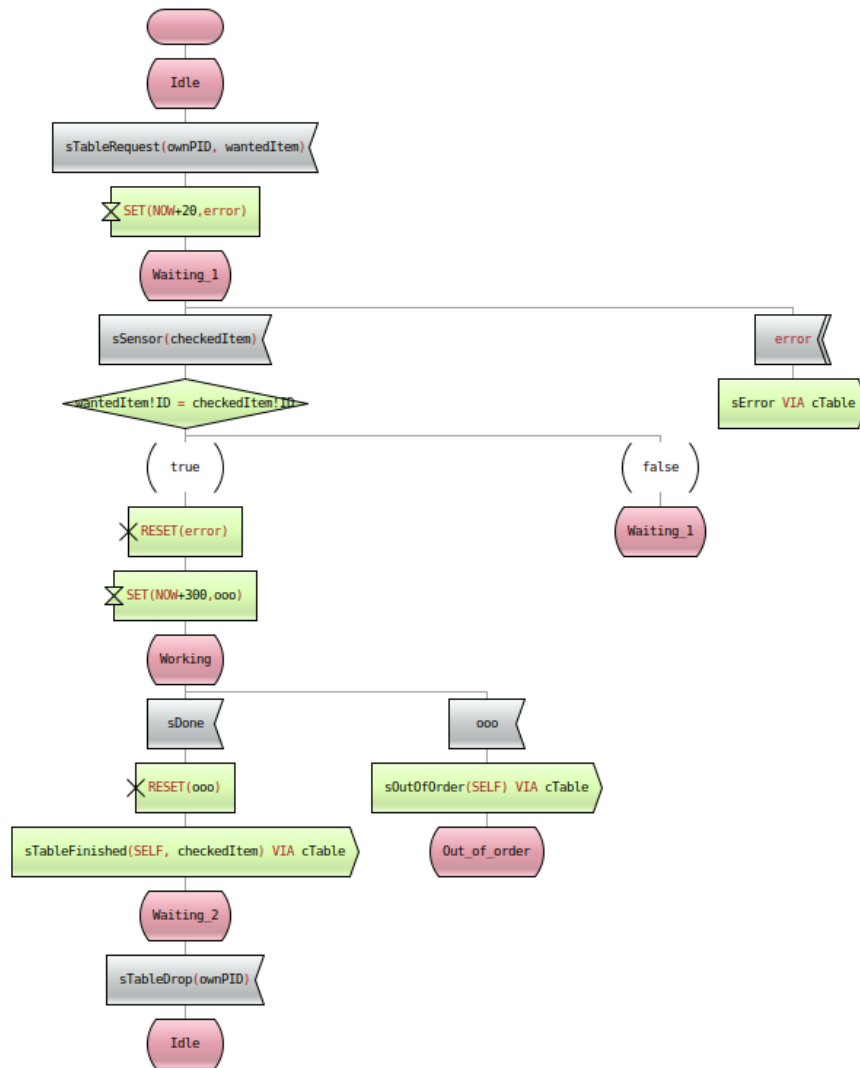


FIGURE 13 – Une table d’usinage

Le process pour les tables d’usinage est quelque peu plus compliqué que ceux des autres robots. Une fois une requête reçue, un timer de 20 secondes est lancé en accord avec la partie 2 de l’énoncé, si la table n’arrive pas à récupérer l’objet d’ici-là, le système entier est mis en défaillance. Sinon, si l’objet est trouvé et pris en charge, nous continuons l’exécution. Un autre timer de 5 min est lancé

pendant que la table continue à travailler sur l'usinage de la pièce. Le signal `sDone` qui doit être manuellement envoyé est donc utilisé afin d'activer ou non l'erreur `ooo` indirectement. Dans ce dernier cas, la table est tout simplement considéré en panne, mais le système entier n'est pas mis en défaillance.

Une fois l'usinage terminé, il ne reste plus qu'à attendre le signal pour déposer la pièce sur le tapis roulant avant de reprendre au début la boucle.

1.4 Simulation

Etant donné les demandes de l'énoncé, il y a plusieurs cas possibles d'exécution du système selon si l'on envoie ou non certains signaux et donc si certains timers se lancent ou non. Outre le fonctionnement nominal, nous verrons les différentes situations possibles en fonction de ces erreurs.

1.4.1 Cas normal

Name	Pid	Sig	SDL state
pCentral	1	0	Idle
pDropoffRobot	2	0	Idle
pPickupRobot	3	0	Idle
pSupervisor	4	0	Idle
pTable	5	0	Idle
pTable	6	0	Idle
pTable	7	0	Idle
pTable	8	0	Idle
pTable	9	0	Idle


```

>Task pTable(6) prio:0 created by pSupervisor(4) at: 0 ticks
>Task pTable(7) prio:0 created by pSupervisor(4) at: 0 ticks
>Task pTable(8) prio:0 created by pSupervisor(4) at: 0 ticks
>Task pTable(9) prio:0 created by pSupervisor(4) at: 0 ticks
>Task pSupervisor(4) has changed to state Idle at: 0 ticks
>Task pTable(5) has changed to state Idle at: 0 ticks
>Task pTable(6) has changed to state Idle at: 0 ticks
>Task pTable(7) has changed to state Idle at: 0 ticks
>Task pTable(8) has changed to state Idle at: 0 ticks
>Task pTable(9) has changed to state Idle at: 0 ticks
>stop
4

```

FIGURE 14 – Simulation Normal, 1ère itération

Suite à la création des différentes tables, le système est initialement à l'arrêt tant qu'il n'y a pas de nouvel objet introduit par l'opérateur avec le signal `sNewGood(Item)`.

Name	Pid	Sig	SDL state
pCentral	1	0	Idle
pDropoffRobot	2	0	Dropping
pPickupRobot	3	0	Idle
pSupervisor	4	0	Waiting_Drop
pTable	5	0	Idle
pTable	6	0	Idle
pTable	7	0	Waiting_1
pTable	8	0	Idle
pTable	9	0	Idle


```

>[param1=7,param2={ID=0,specialization=Fraisage}]
>Task pCentral(1) has changed to state Idle at: 0 ticks
>Signal: sDropoffRequest received by: pDropoffRobot(2) at: 0 ticks
>[param1={ID=0,specialization=Fraisage}]
>Timer: error started by: pDropoffRobot(2) at: 0 ticks
>Task pDropoffRobot(2) has changed to state Dropping at: 0 ticks
>Signal: sTableRequest received by: pTable(7) at: 0 ticks
>[param1=7,param2={ID=0,specialization=Fraisage}]
>Timer: error started by: pTable(7) at: 0 ticks
>Task pTable(7) has changed to state Waiting_1 at: 0 ticks
>stop
>

```

FIGURE 15 – Simulation Normal, 2ème itération

Une fois le signal envoyé, nous nous retrouvons dans une situation où le robot d'alimentation attend le signal du tapis roulant, envoyé manuellement dans ce système simulé, et où la table attend tout simplement que l'objet passe devant elle.

Name	Pid	Sig	SDL state
pCentral	1	0	Idle
pDropoffRobot	2	0	Dropping
pPickupRobot	3	0	Idle
pSupervisor	4	0	Idle
pTable	5	0	Waiting_1
pTable	6	0	Idle
pTable	7	0	Waiting_1
pTable	8	0	Idle
pTable	9	0	Idle


```

>[param1=5,param2={ID=1,specialization=Percege}]
>Task pCentral(1) has changed to state Idle at: 0 ticks
>Signal: sDropoffRequest received by: pDropoffRobot(2) at: 0 ticks
>[param1={ID=1,specialization=Percege}]
>Timer: error started by: pDropoffRobot(2) at: 0 ticks
>Task pDropoffRobot(2) has changed to state Dropping at: 0 ticks
>Signal: sTableRequest received by: pTable(5) at: 0 ticks
>[param1=5,param2={ID=1,specialization=Percege}]
>Timer: error started by: pTable(5) at: 0 ticks
>Task pTable(5) has changed to state Waiting_1 at: 0 ticks
>stop
>

```

FIGURE 16 – Simulation Normal, 3ème itération

Nous envoyons donc un signal au robot d'alimentation que le tapis est libre une première fois, puis nous enchaînons par un nouvel objet demandé au superviseur, re-préparant ainsi une autre table pour ce nouvel objet et demandant une nouvelle fois au robot d'alimentation de déposer la pièce.

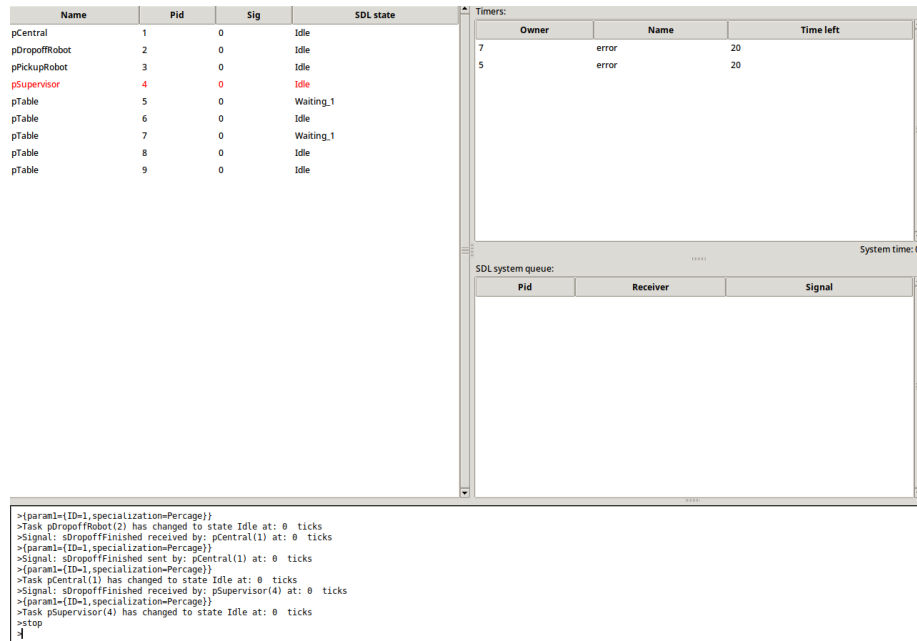


FIGURE 17 – Simulation Normal, 4ème itération

Encore une fois nous envoyons le signal que le tapis roulant est libre et les deux objets sont donc en train de se déplacer à travers les tables. Nous pouvons bien voir à droite que les deux timers sont lancés et qu'il suffit que le temps arrive à 0 pour que la défaillance arrive.

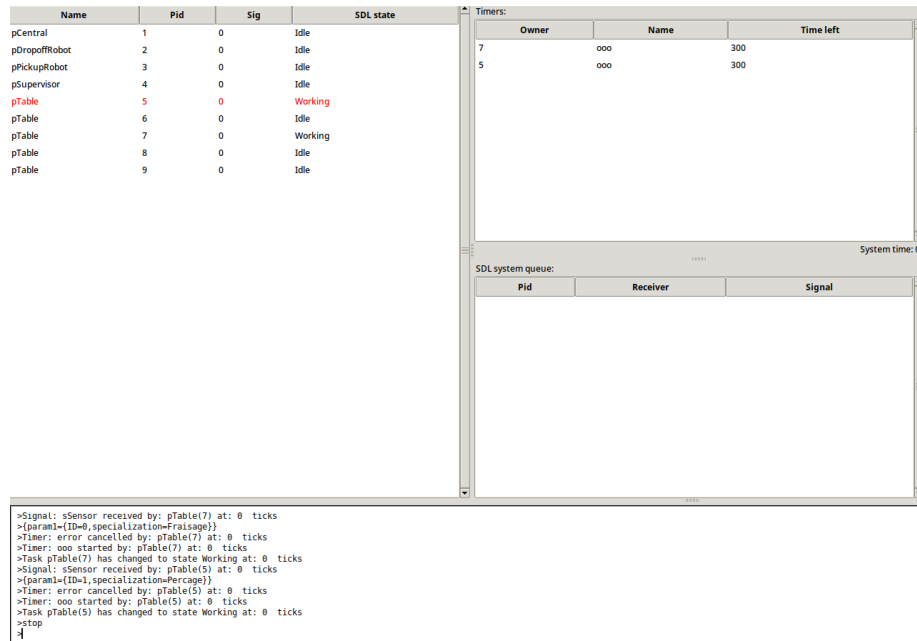


FIGURE 18 – Simulation Normal, 5ème itération

Bien sûr, nous sommes ici dans le fonctionnement nominal, nous envoyons donc aux deux tables les signaux `sSensor(Item)` correspondant bien aux items qu'elles attendent. Les deux timers de 5 min des tables sont eux aussi bien lancés.

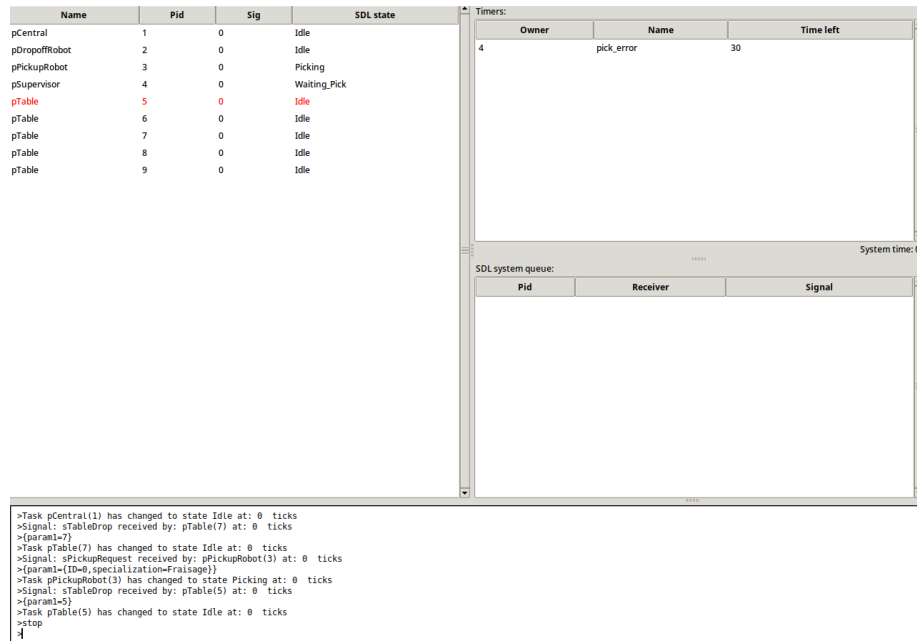


FIGURE 19 – Simulation Normal, 6ème itération

Suite à l'envoi des signaux `sDone`, utilisés à des fins de simulation, les deux tables ont donc fini leur travail et ont alerté le superviseur. Ce dernier envoie aux deux l'ordre de déposer les objets sur le tapis roulant, ce qui est fait. Il ne reste plus que l'objet attendu en premier passe devant le robot de retrait pour qu'il soit récupéré. Par ailleurs, le timer `pick_error` est bien lancé.

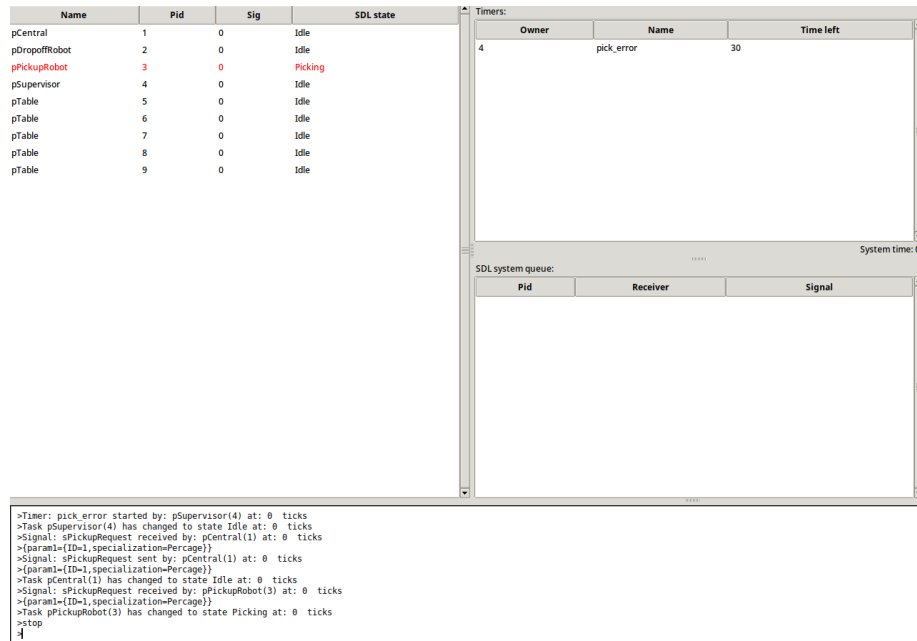


FIGURE 20 – Simulation Normal, 7ème itération

Le premier objet est bien récupéré suite à l'envoi du signal `sSensor(Item)` au robot de retrait et il ne reste plus que le deuxième. Le timer est aussi relancé à l'occasion de ce nouvel objet.

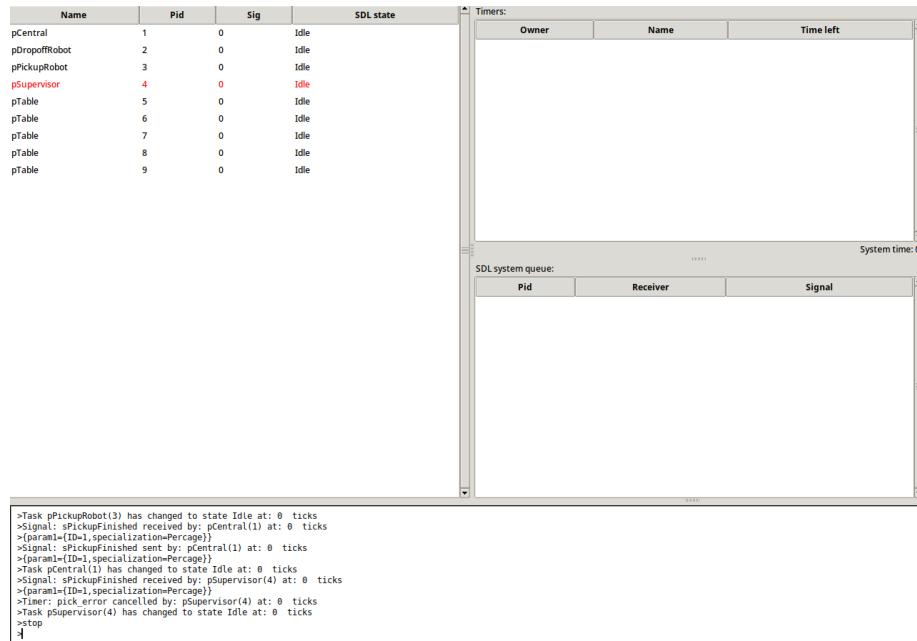


FIGURE 21 – Simulation Normal, 8ème itération

Enfin, les deux pièces à usiner ont été complètement traitées suite aux différentes opérations du système. Il nous reste encore à voir les cas où tout ne se passe pas comme prévu...

1.4.2 Erreur d'alimentation

Si dans les 30 secondes suivant la request du superviseur, le robot d'alimentation ne dépose pas l'objet demandé, le système entier doit être mis en défaillance.

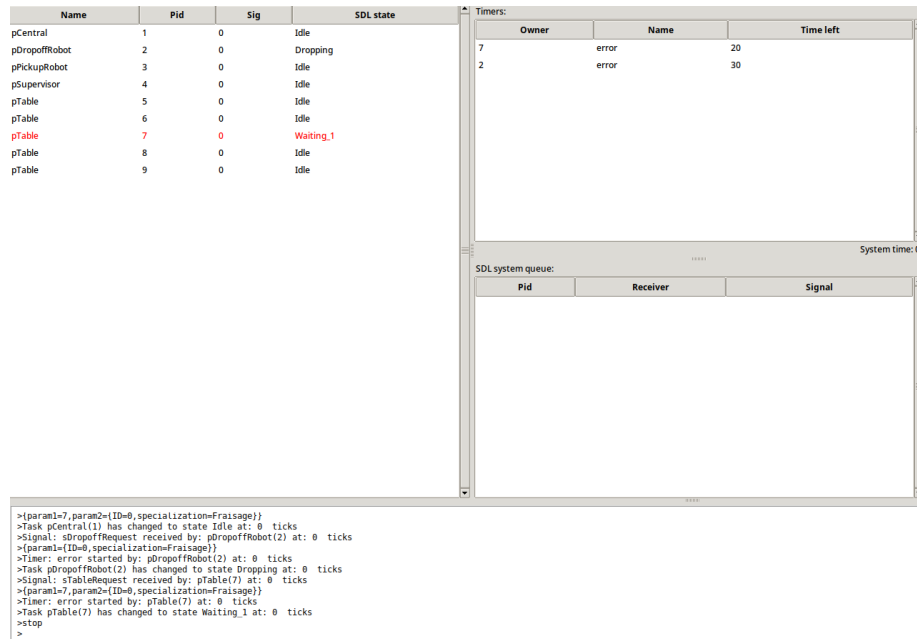


FIGURE 22 – Simulation Dropoff, 1ère itération

Un premier objet est donc demandé dans le cadre de cette simulation. La table se prépare et le robot d'alimentation attend de pouvoir placer la pièce sur le tapis roulant...

Name	Pid	Sig	SDL state
pCentral	1	0	Idle
pDropoffRobot	2	0	Dropping
pPickupRobot	3	0	Idle
pSupervisor	4	0	Out_of_order
pTable	5	0	Idle
pTable	6	0	Idle
pTable	7	0	Waiting_1
pTable	8	0	Idle
pTable	9	0	Idle

Owner	Name	Time left
2	error	0

Pid	Receiver	Signal
-----	----------	--------


```

>stop
>Forcing timer: error started by: (2) to go off
>run
>Timer: error received by: pTable(7) at: 30 ticks
>Signal: sError sent by: pTable(7) at: 30 ticks
>Signal: sError received with priority by: pCentral(1) at: 30 ticks
>Signal: sError sent by: pCentral(1) at: 30 ticks
>Task pCentral(1) has changed to state Idle at: 30 ticks
>Signal: sError received with priority by: pSupervisor(4) at: 30 ticks
>Task pSupervisor(4) has changed to state Out of order at: 30 ticks
>stop
-1

```

FIGURE 23 – Simulation Dropoff, 2ème itération

Or cette chance n'arrivera jamais, dans ce cas-là un signal `sError` est remonté jusqu'au superviseur qui se met en défaillance. Il est donc impossible que le système se remette à fonctionner dans cet état-là.

1.4.3 Erreur d'usage

Si dans les 20 secondes suivant la requête du superviseur à une table, une table ne récupère pas l'objet demandé, le système doit être mis en défaillance. Pour des questions de synthèse, nous nous attarderons désormais plus sur les étapes telles que l'envoi d'un nouvel objet ou encore l'envoi du signal `sConveyorClear` si ce n'est pas le focus du cas d'erreur actuel.

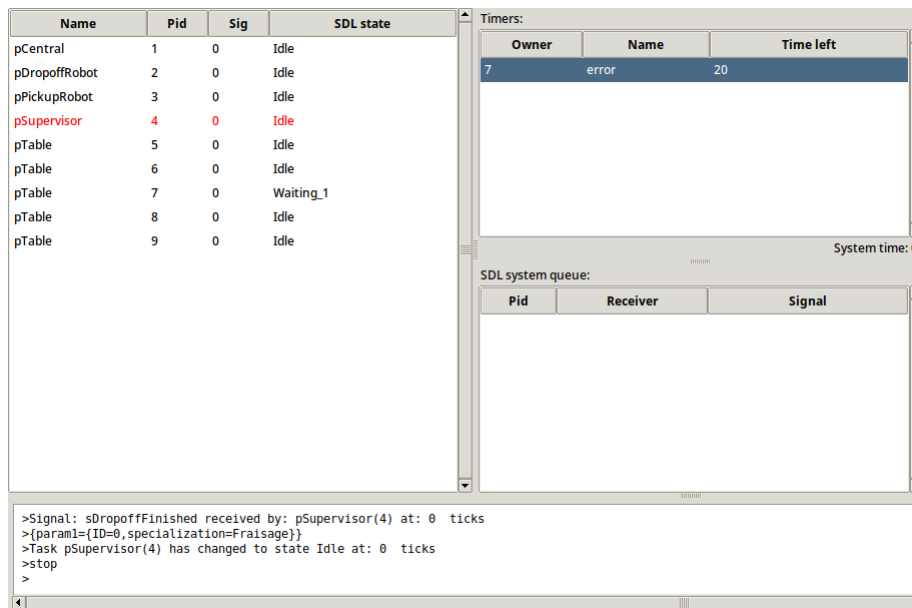


FIGURE 24 – Simulation Usinage, 1ère itération

Une fois l'objet en "marche" sur le tapis roulant, la table d'usinage est censée récupérer l'objet à la fin. Or que se passe-t-il si 20 secondes passent sans que le capteur de la table ne repère son objet...

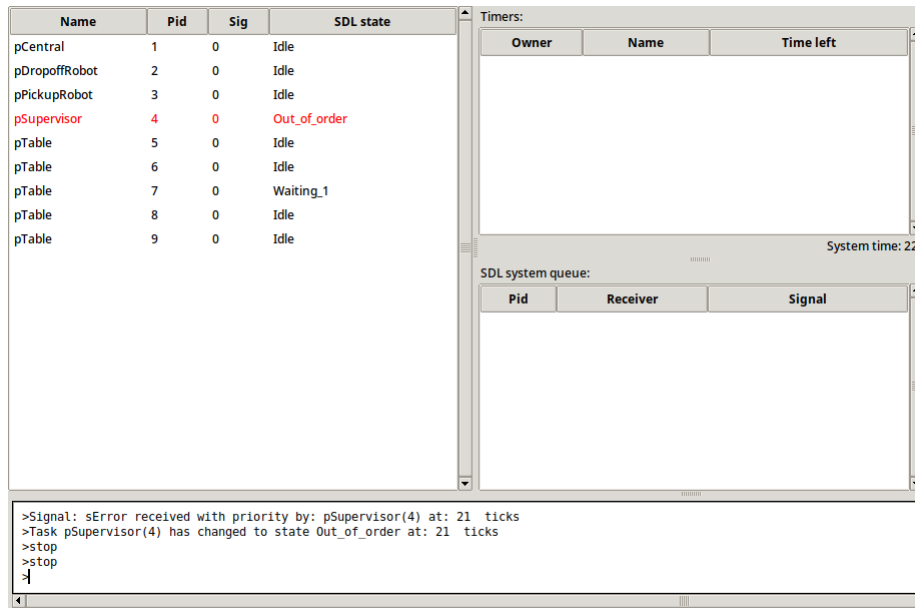


FIGURE 25 – Simulation Usinage, 2ème itération

Le superviseur se met là aussi en défaillance et il est donc impossible de continuer à faire fonctionner le système.

1.4.4 Machine en panne

Si après 5 min de travail sur une pièce, une table n'a toujours pas finie, elle doit se déclarer comme en panne auprès du superviseur.

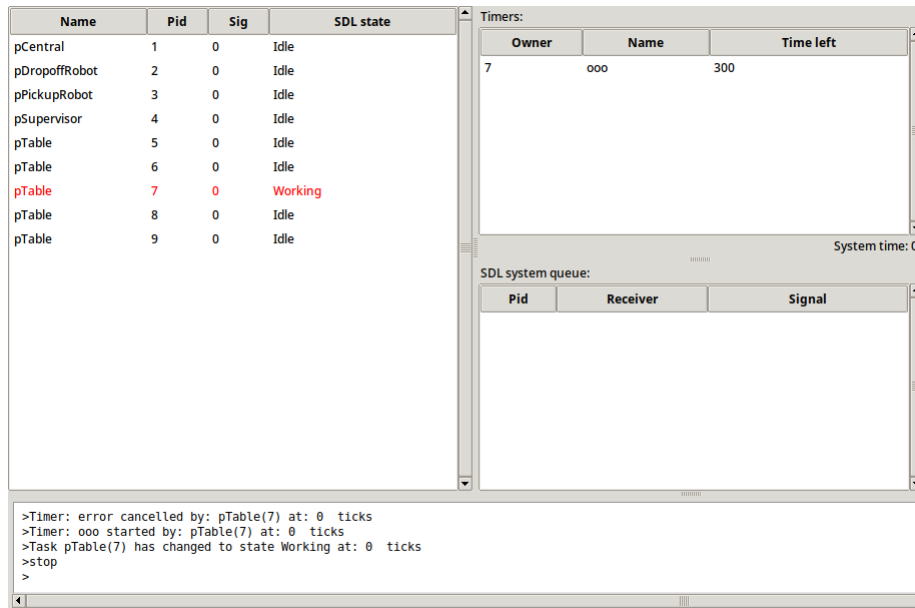


FIGURE 26 – Simulation Panne, 1ère itération

Nous sommes donc au moment où une table est en train de travailler sur sa pièce, essayons de voir ce qu'il se passe si 5 min passent sans aucune nouvelle de la table, et que le signal `sDone` n'arrive jamais à destination...

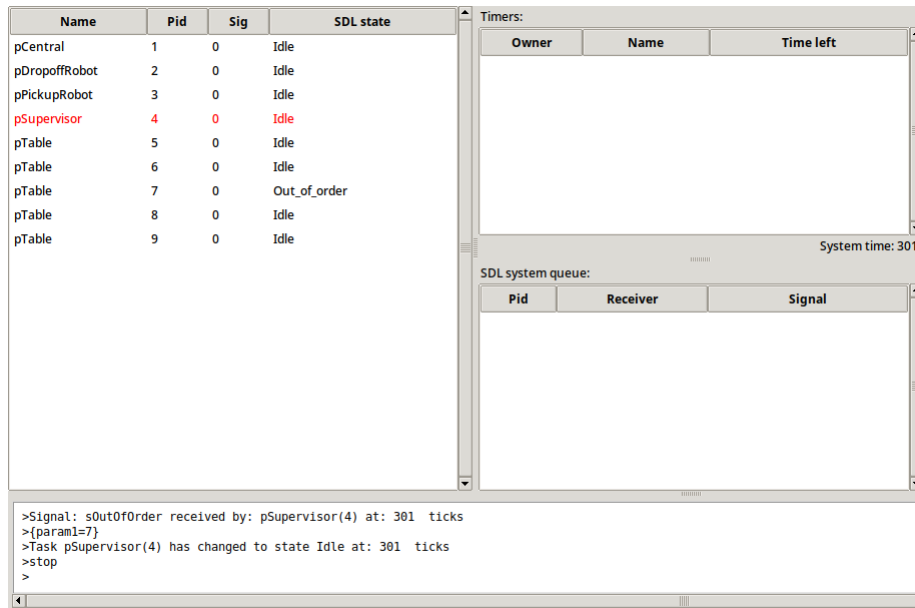


FIGURE 27 – Simulation Panne, 2ème itération

La table se déclare effectivement en panne, et il est impossible au superviseur d’y attribuer de nouvelles pièces de par l’array `occupied` qui n’a pas bougé, et ne bougera pas concernant cette table.

1.4.5 Erreur de retrait

Si après 30 secondes suivant la request du superviseur au robot de retrait, ce dernier ne récupère pas l’objet, le système entier doit être mis en défaillance.

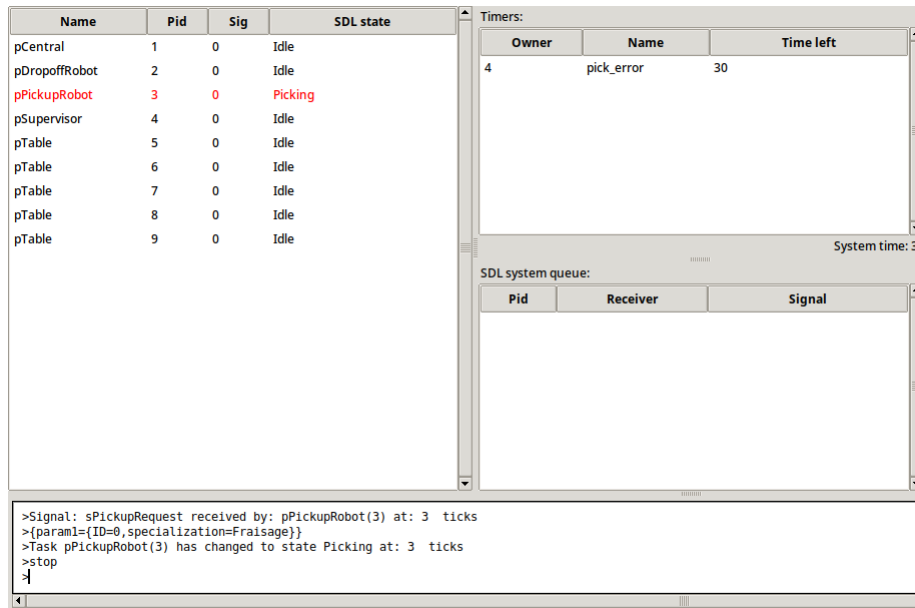


FIGURE 28 – Simulation Retrait, 1ère itération

Nous sommes donc après le dépôt d'une pièce par une table sur le tapis roulant, là où le robot de retrait attend de pouvoir prendre l'objet attendu pour le ranger. Que se passe-t-il si 30 secondes passent sans que rien ne soit envoyé au robot...

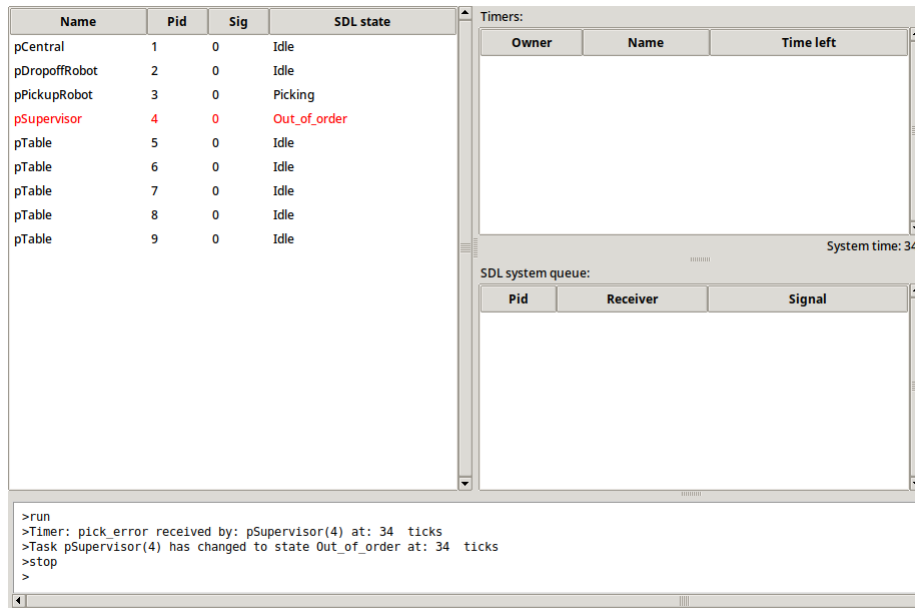


FIGURE 29 – Simulation Retrait, 2ème itération

Comme prévu, le superviseur se met en défaillance et le système est donc dans l'impossibilité de continuer à travailler.

1.5 Conclusion

Nous avons donc respecté chacun des engagements concernant les erreurs de la partie 2 de l'énoncé, et nous avons un système capable de gérer plusieurs pièces en même temps. Certes, de nombreuses interactions entre utilisateur et système sont nécessaires, mais cela est dû par les contraintes imposés par les timers et le fait que certains éléments ne sont pas déclarés dans ce système en tant que process, comme le tapis roulant ou les différents capteurs par exemple. Par ailleurs, un moyen plus efficace pour les recheck de tables pourrait être mis en place. Malgré tout, la logique reste globalement correcte par rapport à l'énoncé.

2 Ordonnancement Temps Réel

2.1 Introduction

Dans cette section, nous analysons les aspects temps réel de la cellule de fabrication en utilisant différents modes d'ordonnancement. Le simulateur fonctionne sur un processeur unique et considère des tâches périodiques et apériodiques avec différentes contraintes (indépendance, précédence, partage de ressources).

2.2 Mode 1 : Tâches Périodiques Indépendantes

2.2.1 Présentation

Ce mode considère uniquement des tâches périodiques indépendantes correspondant à différents types de pièces usinées. Chaque tâche périodique est définie par :

- P_i : la période de la tâche.
- C_i : le temps d'exécution maximal de la tâche.

2.2.2 Ensembles de Tâches

Les ensembles de tâches testés dans ce mode sont présentés dans le tableau suivant :

TABLE 1 – Ensembles de Tâches Périodiques

Ensemble	Tâche (i)	Période (P_i)	Temps d'exécution (C_i)
Exemple 1	T_1	8	2
	T_2	10	2
	T_3	8	1
	T_4	16	2
Exemple 2	T_1	8	3
	T_2	16	3
	T_3	4	1
	T_4	8	1
Exemple 3	T_1	6	2
	T_2	9	3
	T_3	15	3
	T_4	8	2
Exemple 4	T_1	6	2
	T_2	12	4
	T_3	24	4
	T_4	6	1

2.2.3 Ordonnancement avec RM

Calcul de la charge cumulative (CS) pour l'ordonnancement RM pour l'ensemble 1 :

Le calcul de la charge cumulative (**CS**) permet de vérifier si l'ensemble des tâches est ordonnable sous l'algorithme RM. La condition générale est donnée par l'inégalité suivante :

$$CS = \sum_{i=1}^n \frac{C_i}{P_i}$$

où :

- C_i est le coût d'exécution (ou le temps d'exécution) de la tâche i ,

- P_i est la période de la tâche i ,
- n est le nombre total de tâches.

Pour qu'un ensemble de tâches soit ordonnançable, la charge cumulative doit satisfaire la condition suivante :

$$CS \leq n \times (2^{1/n} - 1)$$

Exemple de calcul sur l'exemple 1

Considérons un ensemble de tâches avec les caractéristiques suivantes :

Tâche (i)	P_i	C_i
1	8	2
2	8	1
3	10	2
4	16	2

Étape 1 : Calcul de la charge cumulative (CS)

$$CS = \frac{C_1}{P_1} + \frac{C_2}{P_2} + \frac{C_3}{P_3} + \frac{C_4}{P_4}$$

$$CS = \frac{2}{8} + \frac{1}{8} + \frac{2}{10} + \frac{2}{16}$$

$$CS = 0.2500 + 0.1250 + 0.2000 + 0.1250 = 0.7000$$

Étape 2 : Calcul de la borne d'ordonnançabilité La borne pour $n = 4$ tâches est donnée par :

$$\text{Bound} = 4 \times (2^{1/4} - 1)$$

$$\text{Bound} = 4 \times (1.1892 - 1) = 4 \times 0.1892 = 0.7568$$

Étape 3 : Vérification de l'ordonnançabilité

$$CS = 0.7000 \leq \text{Bound} = 0.7568$$

Ainsi, l'ensemble des tâches est ordonnançable sous l'algorithme RM.

Conclusion

Le calcul de la charge cumulative (CS) montre que les tâches satisfont la condition d'ordonnançabilité, car la charge totale $CS = 0.7000$ est inférieure à la borne théorique 0.7568.

2.2.4 Calcul de la Charge Cumulative (CS) pour Exemple 2

La charge cumulative (**CS**) est utilisée pour vérifier si les tâches périodiques d'un ensemble respectent la condition d'ordonnançabilité. Pour l'ensemble **Exemple 2**, nous avons les tâches suivantes :

Tâche (i)	Période (P_i)	Temps d'exécution (C_i)
T_1	8	3
T_2	16	3
T_3	4	1
T_4	8	1

Étape 1 : Calcul de la charge cumulative (CS) La charge cumulative est donnée par la formule suivante :

$$CS = \sum_{i=1}^n \frac{C_i}{P_i}$$

En appliquant les valeurs des tâches de l'ensemble **Exemple 2** :

$$CS = \frac{C_1}{P_1} + \frac{C_2}{P_2} + \frac{C_3}{P_3} + \frac{C_4}{P_4}$$

$$CS = \frac{3}{8} + \frac{3}{16} + \frac{1}{4} + \frac{1}{8}$$

$$CS = 0.3750 + 0.1875 + 0.2500 + 0.1250 = 0.9375$$

Étape 2 : Calcul de la borne d'ordonnançabilité (Bound) Pour $n = 4$ tâches, la borne théorique est donnée par :

$$Bound = n \times (2^{1/n} - 1)$$

En calculant pour $n = 4$:

$$Bound = 4 \times (2^{1/4} - 1) = 4 \times (1.1892 - 1) = 4 \times 0.1892 = 0.7568$$

Étape 3 : Vérification de l'ordonnançabilité La condition d'ordonnançabilité est satisfaite si :

$$CS \leq Bound$$

En comparant :

$$CS = 0.9375 \quad > \quad Bound = 0.7568$$

2.2.5 Calculs CNS pour l'Ordonnabilité RM

Le critère CNS (Critical Necessary Schedulability) permet de vérifier si un ensemble de tâches est ordonnable selon l'algorithme Rate Monotonic (RM). La condition est donnée par :

$$\forall i, 1 \leq i \leq n, \min_{(k,m) \in R_i} \left\{ \frac{1}{mP_k} \sum_{j=1}^i C_j \left\lceil \frac{mP_k}{P_j} \right\rceil \right\} \leq 1$$

où R_i représente les combinaisons possibles pour chaque tâche i .

Tâche 1 :

- Période : $P_1 = 4$
- Temps d'exécution : $C_1 = 1$

$$R_1 = \left\{ (k, m) \mid 1 \leq k \leq 1, m = 1, \dots, \left\lceil \frac{P_1}{P_k} \right\rceil \right\}$$

Pour $(k = 1, m = 1) : P_k = 4, \sum C_j = 1, \text{ Valeur de la condition} = 0.2500$

Condition minimale pour Tâche 1 : 0.2500

Tâche 2 :

- Période : $P_2 = 8$
- Temps d'exécution : $C_2 = 3$

$$R_2 = \left\{ (k, m) \mid 1 \leq k \leq 2, m = 1, \dots, \left\lceil \frac{P_2}{P_k} \right\rceil \right\}$$

Pour $(k = 1, m = 1) : P_k = 4, \sum C_j = 4, \text{ Valeur de la condition} = 1.0000$

Pour $(k = 1, m = 2) : P_k = 8, \sum C_j = 5, \text{ Valeur de la condition} = 0.6250$

Pour $(k = 2, m = 1) : P_k = 8, \sum C_j = 5, \text{ Valeur de la condition} = 0.6250$

Condition minimale pour Tâche 2 : 0.6250

Tâche 3 :

- Période : $P_3 = 8$
- Temps d'exécution : $C_3 = 1$

$$R_3 = \left\{ (k, m) \mid 1 \leq k \leq 3, m = 1, \dots, \left\lceil \frac{P_3}{P_k} \right\rceil \right\}$$

Pour $(k = 1, m = 1) : P_k = 4, \sum C_j = 5, \text{ Valeur de la condition} = 1.2500$

Pour $(k = 1, m = 2) : P_k = 8, \sum C_j = 6, \text{ Valeur de la condition} = 0.7500$

Pour $(k = 2, m = 1) : P_k = 8, \sum C_j = 6, \text{ Valeur de la condition} = 0.7500$

Pour $(k = 3, m = 1) : P_k = 8, \sum C_j = 6, \text{ Valeur de la condition} = 0.7500$

Condition minimale pour Tâche 3 : 0.7500

Tâche 4 :

- Période : $P_4 = 16$
- Temps d'exécution : $C_4 = 3$

$$R_4 = \left\{ (k, m) \mid 1 \leq k \leq 4, m = 1, \dots, \left\lceil \frac{P_4}{P_k} \right\rceil \right\}$$

Pour $(k = 1, m = 1) : P_k = 4, \quad \sum C_j = 8, \quad \text{Valeur de la condition} = 2.0000$

Pour $(k = 1, m = 2) : P_k = 8, \quad \sum C_j = 9, \quad \text{Valeur de la condition} = 1.1250$

Pour $(k = 1, m = 4) : P_k = 16, \quad \sum C_j = 15, \quad \text{Valeur de la condition} = 0.9375$

Condition minimale pour Tâche 4 : 0.9375

2.2.6 Conclusion

Toutes les tâches satisfont la condition CNS, et donc l'ensemble est ordonnançable selon l'algorithme RM.

2.3 Ordonnancement avec EDF**2.3.1 Présentation**

L'algorithme **Earliest Deadline First (EDF)** est une méthode d'ordonnancement dynamique où les tâches sont ordonnancées en fonction de leur date limite la plus proche. Dans notre cas, la période de chaque tâche (P_i) est égale à sa date limite relative (D_i), c'est-à-dire $P = D$. EDF garantit l'ordonnançabilité si la charge processeur totale (U) satisfait la condition suivante :

$$U = \sum_{i=1}^n \frac{C_i}{P_i} \leq 1$$

où :

- C_i est le temps d'exécution de la tâche i ,
- P_i est à la fois la période et la date limite relative de la tâche i ,
- n est le nombre total de tâches.

2.3.2 Calcul pour l'Exemple 1

L'ensemble des tâches de l'exemple 1 est défini comme suit :

Tâche (i)	Période (P_i)	Temps d'exécution (C_i)
T_1	8	2
T_2	10	2
T_3	8	1
T_4	16	2

Étape 1 : Calcul de la charge processeur

$$U = \sum_{i=1}^4 \frac{C_i}{P_i} = \frac{2}{8} + \frac{2}{10} + \frac{1}{8} + \frac{2}{16}$$

$$U = 0.2500 + 0.2000 + 0.1250 + 0.1250 = 0.7000$$

Étape 2 : Vérification de la condition EDF La charge processeur totale $U = 0.7000$ est inférieure ou égale à 1. Par conséquent, l'ensemble des tâches **Exemple 1** est **ordonnançable** avec EDF.

2.3.3 Calcul pour l'Exemple 2

L'ensemble des tâches de l'exemple 2 est défini comme suit :

Tâche (i)	Période (P_i)	Temps d'exécution (C_i)
T_1	8	3
T_2	16	3
T_3	4	1
T_4	8	1

Étape 1 : Calcul de la charge processeur

$$U = \sum_{i=1}^4 \frac{C_i}{P_i} = \frac{3}{8} + \frac{3}{16} + \frac{1}{4} + \frac{1}{8}$$

$$U = 0.3750 + 0.1875 + 0.2500 + 0.1250 = 0.9375$$

Étape 2 : Vérification de la condition EDF La charge processeur totale $U = 0.9375$ est inférieure ou égale à 1. Par conséquent, l'ensemble des tâches **Exemple 2** est **ordonnançable** avec EDF.

2.4 Mode 2 : Tâches Périodiques et Apériodiques

2.4.1 Présentation

Ce mode ajoute des tâches apériodiques soumises de manière asynchrone. Un serveur est utilisé pour gérer ces tâches. Les tâches périodiques et apériodiques sont définies comme suit :

TABLE 2 – Tâches Périodiques et Apériodiques

Type de Tâche	Tâche (i)	$(P_i) / (A_i)$	Temps d'exécution (C_i)
Périodique	T_1	8	2
	T_2	16	2
	T_3	8	1
	T_4	8	1
Apériodique	T_{ap1}	3	2
	T_{ap2}	4	5

2.4.2 Serveur à Scrutation

Le **serveur à scrutation** est une méthode utilisée pour gérer les tâches apériodiques dans un système temps réel. Cette approche alloue un temps d'exécution fixe et périodique aux tâches apériodiques, en utilisant un serveur périodique.

Principe de fonctionnement Un serveur à scrutation fonctionne comme une tâche périodique, avec une période (P_s) et une capacité (C_s) définies. La capacité du serveur est utilisée pour exécuter les tâches apériodiques qui arrivent dans le système. Si aucune tâche apériodique n'est présente, le serveur reste inactif.

Hypothèses Dans cette étude :

- Les tâches périodiques sont les mêmes que celles définies dans le tableau 2.
- Le serveur à scrutation a une période de $P_s = 16$ et une capacité de $C_s = 4$.
- Les tâches apériodiques sont définies comme suit :

Tâche Apériodique	Instant d'Arrivée (A_i)	Temps d'Exécution (C_i)
T_{ap1}	3	2
T_{ap2}	4	5

Calcul de la charge processeur totale La charge processeur totale (U) est la somme des charges des tâches périodiques et du serveur :

$$U = \sum_{i=1}^n \frac{C_i}{P_i} + \frac{C_s}{P_s}$$

En utilisant les valeurs fournies :

$$U = \frac{2}{8} + \frac{2}{16} + \frac{1}{8} + \frac{1}{8} + \frac{4}{16}$$

$$U = 0.2500 + 0.1250 + 0.1250 + 0.1250 + 0.2500 = 0.8750$$

Vérification de la condition d'ordonnabilité La condition d'ordonnabilité est satisfaite si :

$$U \leq 1$$

Dans notre cas, $U = 0.8750$, ce qui est inférieur

2.5 Mode 3 : Contraintes de Précédence

2.5.1 Présentation

Les tâches suivent des contraintes d'ordre entre elles, définies par un graphe de précédence. Ces contraintes spécifient qu'une tâche ne peut commencer que si toutes les tâches qui la précèdent dans le graphe ont été exécutées. Cela reflète des dépendances logiques ou de production entre les différentes étapes.

Hypothèses et Modèle Dans ce mode, deux groupes de tâches périodiques sont considérés, et un ordre précis est imposé entre les tâches. Les contraintes de précédence à respecter sont :

$$TP_{1,j} \rightarrow TP_{2,j} \rightarrow TP_{3,j} \rightarrow TP_{4,j} \quad (j = 1, 2)$$

où $TP_{i,j}$ est la tâche i du groupe j . Chaque tâche est définie par sa période (P), sa date de réveil (r), et son temps d'exécution (C).

Tâches périodiques considérées Les caractéristiques des tâches périodiques pour les deux groupes sont données dans les tableaux suivants :

TABLE 3 – Groupe 1 : Tâches Périodiques

Tâche (i)	Période (P_i)	Date de réveil (r_i)	Temps d'exécution (C_i)
$TP_{1,1}$	20	1	2
$TP_{2,1}$	20	0	3
$TP_{3,1}$	20	2	4
$TP_{4,1}$	20	0	2

TABLE 4 – Groupe 2 : Tâches Périodiques

Tâche (i)	Période (P_i)	Date de réveil (r_i)	Temps d'exécution (C_i)
$TP_{1,2}$	30	0	4
$TP_{2,2}$	30	2	3
$TP_{3,2}$	30	0	4
$TP_{4,2}$	30	3	4

2.5.2 Ordonnancement avec RM

Calcul pour le Groupe 1 La charge processeur totale est calculée comme suit :

$$U = \sum_{i=1}^4 \frac{C_i}{P_i} = \frac{2}{20} + \frac{3}{20} + \frac{4}{20} + \frac{2}{20}$$

$$U = 0.1 + 0.15 + 0.2 + 0.1 = 0.55$$

La borne d'ordonnançabilité pour $n = 4$ tâches est :

$$\text{Bound} = 4 \times (2^{1/4} - 1) = 4 \times 0.1892 = 0.7568$$

Comme $U = 0.55 \leq \text{Bound} = 0.7568$, l'ensemble des tâches du Groupe 1 est **ordonnançable** sous RM.

Calcul pour le Groupe 2 La charge processeur totale est calculée comme suit :

$$U = \sum_{i=1}^4 \frac{C_i}{P_i} = \frac{4}{30} + \frac{3}{30} + \frac{4}{30} + \frac{4}{30}$$

$$U = 0.1333 + 0.1 + 0.1333 + 0.1333 = 0.5$$

La borne d'ordonnançabilité pour $n = 4$ tâches reste :

$$\text{Bound} = 0.7568$$

Comme $U = 0.5 \leq \text{Bound} = 0.7568$, l'ensemble des tâches du Groupe 2 est également **ordonnançable** sous RM.

2.5.3 Ordonnancement avec EDF

Calcul pour le Groupe 1 La charge processeur totale est calculée comme suit :

$$U = \sum_{i=1}^4 \frac{C_i}{P_i} = \frac{2}{20} + \frac{3}{20} + \frac{4}{20} + \frac{2}{20}$$

$$U = 0.1 + 0.15 + 0.2 + 0.1 = 0.55$$

Comme $U = 0.55 \leq 1$, l'ensemble des tâches du Groupe 1 est **ordonnançable** sous EDF.

Calcul pour le Groupe 2 La charge processeur totale est calculée comme suit :

$$U = \sum_{i=1}^4 \frac{C_i}{P_i} = \frac{4}{30} + \frac{3}{30} + \frac{4}{30} + \frac{4}{30}$$

$$U = 0.1333 + 0.1 + 0.1333 + 0.1333 = 0.5$$

Comme $U = 0.5 \leq 1$, l'ensemble des tâches du Groupe 2 est **ordonnançable** sous EDF.

2.6 Mode 4 : Partage de Ressources

2.6.1 Présentation

Dans ce mode, les tâches périodiques nécessitent l'accès à des ressources partagées (machines d'usinage) pour effectuer leur traitement. Ces ressources doivent être utilisées en exclusivité par une seule tâche à la fois, ce qui nécessite des mécanismes d'exclusion mutuelle pour éviter les conflits. Deux protocoles couramment utilisés pour gérer ces accès sont :

- Le protocole d'héritage de priorité (PIP - Priority Inheritance Protocol).
- Le protocole de plafond de priorité (PCP - Priority Ceiling Protocol).

Tâches et Ressources Considérées Les tâches périodiques accèdent à plusieurs ressources partagées définies par leur période, leur date de réveil, et leurs sections critiques. Les caractéristiques des tâches sont données dans le tableau suivant :

TABLE 5 – Tâches Périodiques avec Partage de Ressources

Tâche (i)	(P_i)	(r_i)	(C_i)	Ressources et Durées
T_1	20	1	6	$R_1(1) \rightarrow \{R_1(1) \parallel R_2(1)\} \rightarrow \{R_3(1) \parallel R_4(1)\}$
T_2	30	2	5	$\{R_4(2) \parallel R_1(2)\} \rightarrow R_1(2)$
T_3	40	1	7	$R_3(3) \rightarrow \{R_4(2) \parallel R_2(3)\} \rightarrow \{R_3(1) \parallel R_4(2)\}$
T_4	50	0	6	$R_2(1) \rightarrow \{R_1(2) \parallel R_2(2) \parallel R_3(2)\} \rightarrow \{R_3(1) \parallel R_4(2)\}$

Gestion des Conflits d'Accès aux Ressources

Protocole d'Héritage de Priorité (PIP) Le protocole PIP fonctionne comme suit :

- Lorsqu'une tâche basse priorité T_L verrouille une ressource et qu'une tâche haute priorité T_H en a besoin, T_L hérite temporairement de la priorité de T_H .
- Cela réduit le risque d'inversion de priorité, mais n'élimine pas complètement les interblocages.

Exemple d'Interblocage sous PIP Dans notre exemple, un interblocage se produit lorsque :

- T_1 verrouille R_1 et entre en attente pour R_2 .
- T_4 verrouille R_2 et demande R_1 .

Dans ce cas, ni T_1 ni T_4 ne peuvent progresser, entraînant un **échec de l'ordonnancement**. Cet échec montre que le protocole PIP peut être insuffisant pour gérer les situations d'interblocage complexes.

Protocole de Plafond de Priorité (PCP) Le protocole PCP évite les inversions de priorité et les interblocages en associant un plafond de priorité à chaque ressource. Les principales étapes sont :

- Une tâche peut verrouiller une ressource uniquement si sa priorité est strictement supérieure au plafond de toutes les ressources actuellement verrouillées.
- Cela garantit qu’aucune tâche basse priorité ne bloque une tâche haute priorité de manière indéfinie.

2.7 Analyse Assistée

2.7.1 Présentation de l’Outil

L’analyse assistée a été réalisée à l’aide de l’outil **Cheddar**, un logiciel d’analyse temps réel. Cheddar est conçu pour vérifier l’ordonnançabilité des tâches dans des systèmes embarqués temps réel. Ses principales fonctionnalités incluent :

- La modélisation des tâches périodiques, apériodiques, et des serveurs.
- L’application des principaux algorithmes d’ordonnancement, tels que RM et EDF.
- La gestion des ressources partagées via des protocoles comme PIP et PCP.
- La génération de graphiques et de rapports détaillés sur l’exécution des tâches.

Les tests ont été configurés dans Cheddar pour reproduire les scénarios définis dans les analyses manuelles. Chaque mode (1 à 4) a été modélisé en utilisant les périodes, temps d’exécution, et caractéristiques des tâches, conformément aux spécifications.

2.7.2 Résultats de l’Analyse

Les résultats obtenus à l’aide de Cheddar sont globalement cohérents avec ceux des analyses manuelles, avec les observations suivantes :

- Les modes 1, 3, et 4 ont produit des résultats identiques à ceux calculés manuellement, confirmant l’ordonnançabilité des tâches dans ces cas.
- Une divergence a été observée pour le mode 2 avec un serveur à scrutation :
 - L’ordonnancement produit par Cheddar diffère légèrement des résultats manuels.
 - Cette différence est probablement due à une variation dans l’implémentation du serveur à scrutation dans Cheddar, qui pourrait gérer différemment la répartition du temps processeur ou les priorités des tâches.

Ces résultats montrent l’intérêt de l’utilisation d’outils comme Cheddar pour valider les analyses manuelles, tout en mettant en évidence les limites potentielles liées aux différences d’implémentation.

2.8 Conclusion

L'étude des différents modes d'ordonnancement a permis d'explorer et de valider les concepts essentiels pour la gestion des tâches dans des systèmes temps réel. Les points suivants ont été mis en évidence :

- **EDF (Earliest Deadline First)** s'est montré plus flexible et performant dans la majorité des cas, notamment lorsqu'il s'agit de gérer des contraintes complexes comme celles de précédence ou l'intégration de tâches apériodiques.
- **RM (Rate Monotonic)** est efficace pour des systèmes relativement simples où les tâches ont des périodes fixes, mais ses limitations sont apparentes dans des cas plus complexes, notamment lorsque la charge processeur dépasse les seuils d'ordonnancabilité.
- **Gestion des ressources partagées** : Les protocoles tels que le PCP (Priority Ceiling Protocol) sont indispensables pour prévenir les interblocages et garantir un accès ordonné aux ressources partagées, comme démontré dans le mode 4.
- **Outil Cheddar** : L'utilisation de Cheddar a permis de valider les calculs manuels et d'apporter une analyse complémentaire grâce à des simulations précises. Bien que la majorité des résultats soient cohérents, des divergences mineures, comme pour le serveur à scrutation, montrent l'importance de comprendre les implémentations spécifiques des outils.

Cette analyse met en évidence la nécessité d'adopter une approche méthodique pour garantir l'ordonnancabilité des tâches dans des systèmes temps réel complexes. Le choix de l'algorithme d'ordonnancement et des protocoles de gestion des ressources doit être adapté à la nature des tâches et aux contraintes spécifiques du système étudié. Enfin, l'intégration d'outils comme Cheddar apporte une dimension pratique et fiable à l'évaluation de ces systèmes.

Table des figures

1	Déclarations	4
2	Process et canaux	5
3	Déclarations du superviseur	7
4	Création des tables	8
5	Gestion des nouveaux objets	9
6	Gestion des signaux provenant des robots	10
7	Recheck des tables	11
8	Attente du robot d'alimentation	12
9	Attente du robot de retrait	13
10	Robot de retrait	14
11	Robot d'alimentation	15
12	Bloc de communication central	16
13	Une table d'usinage	17
14	Simulation Normal, 1ère itération	18
15	Simulation Normal, 2ème itération	19
16	Simulation Normal, 3ème itération	20
17	Simulation Normal, 4ème itération	21
18	Simulation Normal, 5ème itération	22
19	Simulation Normal, 6ème itération	23
20	Simulation Normal, 7ème itération	24
21	Simulation Normal, 8ème itération	25
22	Simulation Dropoff, 1ère itération	26
23	Simulation Dropoff, 2ème itération	27
24	Simulation Usinage, 1ère itération	28
25	Simulation Usinage, 2ème itération	29
26	Simulation Panne, 1ère itération	30
27	Simulation Panne, 2ème itération	31
28	Simulation Retrait, 1ère itération	32
29	Simulation Retrait, 2ème itération	33